KeyperVPN

Post-Quantum Secure Overlay VPN

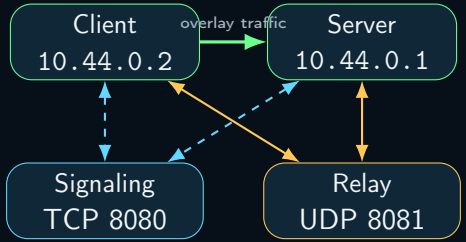
Hybrid ML-KEM-768 + X25519 session establishment, encrypted UDP overlay, constant-size traffic morphing

Problem

Two Linux hosts should communicate as if they share one subnet, while the security model remains visible and defensible.

Core stack

ML-KEM-768, X25519, HKDF-SHA256, ChaCha20-Poly1305, Linux TUN, WebSocket signaling, UDP relay, traffic morphing.



Adith Biju Adwait Mishra Aryan Sharma

Introduction

Why this project?

- Classical VPN key exchange is vulnerable to future quantum attacks.
- Remote systems still need practical, low-friction secure overlay connectivity.
- Encrypted transport should not leak obvious payload-size information.

Threat model

Adversary records encrypted sessions now and attempts retrospective decryption later with stronger compute.

Target outcome

Same-subnet behavior with encrypted overlay transport and visible security evidence in Wireshark.

Objectives

Project objectives

- Create a working overlay VPN interface between two Linux hosts.
- Use hybrid post-quantum + classical key establishment.
- Show encrypted transport with Wireshark-visible ciphertext only.
- Add constant-size packet morphing to reduce payload-size leakage.

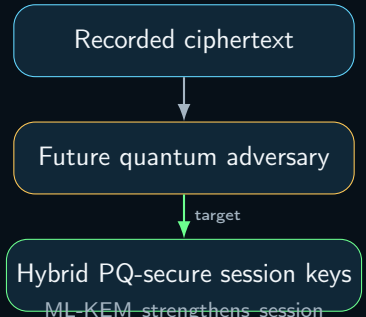
Security stance

Hybrid ML-KEM-768 + X25519 for session establishment, HKDF-SHA256 for key derivation, and ChaCha20-Poly1305 for authenticated encryption.

Quantum Security Motivation

Why post-quantum security matters

- Shor's algorithm threatens elliptic-curve Diffie–Hellman and other classical public-key systems.
- VPN traffic can contain credentials, admin access, and internal service data with long confidentiality lifetimes.
- "Harvest now, decrypt later" means encrypted traffic captured today may remain strategically valuable.



ML-KEM strengthens session establishment against retrospective decryption attempts.

Why hybrid ML-KEM-768 + X25519?

The post-quantum and classical shared secrets are combined through HKDF-SHA256, so the tunnel does not depend on only one cryptographic family.

Literature Survey I

Reference systems

- **WireGuard**: minimal, high-performance VPN with strong modern cryptography, but not post-quantum.
- **Tailscale**: secure overlay with control-plane coordination and relay fallback.
- **Nebula**: scalable overlay with discovery through lighthouse nodes.

System	Overlay	Relay	PQ KEX	Size morphing
WireGuard	No	No	No	No
Tailscale	Yes	Yes	No	No
Nebula	Yes	Partial	No	No
KeyperVPN	Yes	Yes	Yes	Yes

Literature Survey II

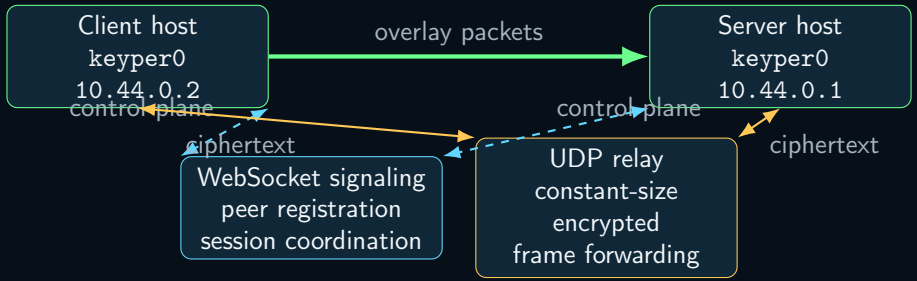
Cryptographic building blocks

- **ML-KEM-768**: NIST-standardized lattice-based KEM.
- **X25519**: efficient classical elliptic-curve Diffie–Hellman.
- **ChaCha20-Poly1305**: AEAD for confidentiality and integrity.

Traffic-analysis defense direction

- BuFLO and traffic morphing aim to reduce packet-size fingerprinting.
- KeyperVPN adopts the practical subset: fixed-size encrypted frames.

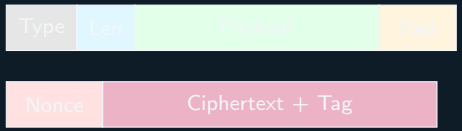
System Design & Architecture



- Overlay networking is performed through a Linux TUN interface.
- Signaling coordinates session formation; relay forwards only encrypted UDP frames.
- Session keys are derived end-to-end and never terminated at the relay.

System Design: Packet & Trust Model

Encrypted frame structure



Trust model

- Signaling sees peer roles and public keys.
- Relay sees only encrypted UDP frames.
- Session keys stay only on client and server peers.
- Traffic morphing hides payload-size differences, though not timing metadata.

Implementation Details I

Modules developed and integrated

- **CryptoEngine**: hybrid key exchange and AEAD encryption
- **SignalingClient**: WebSocket control-plane messaging
- **RelayTransport**: UDP registration and ciphertext forwarding
- **TunDevice**: Linux TUN interface and routing
- **VPNTunnel**: orchestration, state machine, telemetry
- **Ink UI**: runtime dashboard and operator-facing logs

Item	Value
Overlay subnet	`10.44.0.0/24`
Control-plane port	TCP 8080
Relay port	UDP 8081
Default morph size	1200B
Tunnel MTU	1280B

Implementation Details II: Session Establishment

Algorithm

1. Generate ML-KEM-768 and X25519 key pairs.
2. Register peer role and public keys through signaling.
3. Client encapsulates ML-KEM and computes X25519.
4. Server decapsulates ML-KEM and computes X25519.
5. HKDF-SHA256 derives directional session keys.
6. Encrypted overlay enters connected state.

```
const result = await
    initiateKeyExchange(
  this.keys,
  remoteKyber,
  remoteX25519
);
this.sessionKeys = result.sessionKeys;

this.signalingClient?.sendSessionInit(
  message.peer.peerId,
  Buffer.from(result.kyberCiphertext)
    .toString('base64')
);
```

Implementation Details III: Hybrid Crypto Snippet

```
const [ciphertext, kyberShared] =
  await kyberInstance.encap(remoteKyberPK);

const x25519Shared = x25519.getSharedSecret(
  localKeys.x25519PrivateKey,
  remoteX25519PK,
);

const derived = hkdf(
  sha256,
  combinedSecrets,
  undefined,
  info,
  64
);
```

- ML-KEM contributes the post-quantum shared secret.
- X25519 contributes a mature classical shared secret.
- HKDF binds both into directional send/receive keys.

Implementation Details IV: Tunnel Orchestration Snippet

```
if (this.config.noTun === false) {
  this.tun = new TunDevice(
    this.config.tunName,
    this.config.tunAddress,
    this.config.tunNetmask,
    this.config.tunMTU,
  );
  await this.tun.open();
  this.tun.setupRouting(this.config.peerTunAddress);
}
```

What this enables

- creation of `keyper0`
- assignment of overlay IPs
- routing of peer traffic through the encrypted overlay path

Implementation Details V: Operator View

Terminal dashboard

- connection state
- peer ID and overlay IP
- crypto suite and transport mode
- latency, packets, bytes, and live events

```
LINK CONNECTED // fc24aede
crypto       Kyber-768 + X25519
transport    UDP relay overlay
morphing     constant 1200B
peer         d325bc0b
vpn-ip       10.44.0.1
latency      1ms
uplink       3.5KB
downlink     3.5KB
```

Results & Analysis I: Implementation Status

100% implementation outcomes

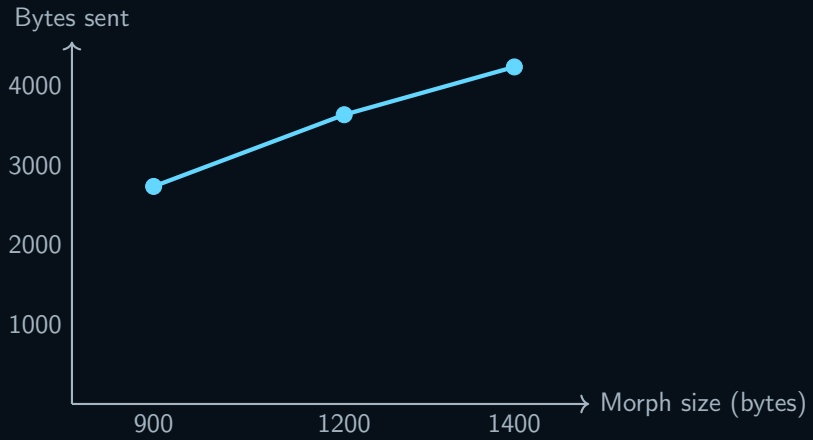
- Signaling server and UDP relay implemented
- Client and server overlay peers implemented
- Hybrid ML-KEM-768 + X25519 handshake implemented
- ChaCha20-Poly1305 encrypted data plane implemented
- Linux TUN creation and peer routing implemented
- Traffic morphing with constant-size frames implemented
- Terminal UI and telemetry implemented

Results & Analysis II: Measured Performance

Morph size	Success	RTT (ms)	Bytes sent	Packets
900B	Yes	4	2731	4
1200B	Yes	1–2	3631	4
1400B	Yes	2	4231	4

- All measured runs reached the connected state successfully.
- Increasing morph size increases traffic overhead but stabilizes visible packet lengths.
- Low localhost RTT shows encryption and relay stages still remain responsive.

Results & Analysis III: Overhead Trend



Interpretation: stronger size flattening increases bandwidth overhead per transmitted event.

Results & Analysis IV: Comparison

Morph size	Relative bytes	Mean pkt size
900B	1.00×	682.8B
1200B	1.33×	907.8B
1400B	1.55×	1057.8B

Key observation

- Functional behavior stayed constant.
- Packet count remained unchanged across runs.
- Only the morphing parameter drove the traffic increase.

Results & Analysis V: Testing Outcomes

Test matrix

- signaling server startup and peer registration
- session matching and relay registration
- hybrid key establishment
- encrypted echo request / reply
- runtime stats collection
- morph-size variation
- clean shutdown and reconnection handling

Observed status

All planned local validation checks passed in the implemented environment.

Results & Analysis VI: Output Verification

Linux-side proof

```
$ ip addr show keyper0
keyper0: <POINTOPOINT,MULTICAST,UP,LOWER_UP>
    inet 10.44.0.2/24

$ ping 10.44.0.1
64 bytes from 10.44.0.1: icmp_seq=1 ...
```

Wireshark-side proof

- Filter: `udp.port == 8081`
- Observe encrypted UDP frames only
- Payload not readable as raw ICMP data
- Packet lengths remain nearly flat with morphing enabled

Challenges & Solutions

Challenge

- Direct NAT traversal is operationally brittle.
- TUN integration requires root privileges and Linux-specific handling.
- Post-quantum integration increases handshake complexity.
- Traffic-analysis resistance introduces bandwidth overhead.

Solution

- Use relay-assisted overlay for deterministic secure forwarding.
- Keep TUN wrapper modular and isolated.
- Use a hybrid KEX design instead of replacing classical crypto entirely.
- Use fixed-size morphing as a practical first-stage defense.

Conclusion & Future Work

Conclusion

KeyperVPN successfully demonstrates a secure overlay VPN with hybrid post-quantum key establishment, authenticated encrypted forwarding, Linux TUN integration, and constant-size traffic morphing.

Future work

- direct NAT traversal with relay fallback
- multi-peer mesh extension
- adaptive morphing beyond fixed-size padding
- larger-scale throughput and loss benchmarking
- hardened identity and authentication layers

References

- NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," FIPS 203, 2024.
- A. Langley, M. Hamburg, and S. Turner, "Elliptic Curves for Security," RFC 7748, 2016.
- Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," RFC 8439, 2018.
- J. A. Donenfeld, "WireGuard: Next Generation Kernel Network Tunnel," NDSS, 2017.
- Tailscale, "How NAT traversal works," `https://tailscale.com/blog/how-nat-traversal-works`
- Slack, "Nebula," `https://github.com/slackhq/nebula`
- C. V. Wright, S. E. Coull, and F. Monrose, "Traffic Morphing," NDSS, 2009.